

Uber

functional requirements:

- user should be able to book a cab based on his preference,
- user should be able to search for any location based on his preference,'
- rider should be able to accept/reject any ride based on his preference,

non-functional requirements:

- data-partitioning,
- load balancing,
- fault tolerance,
- vertical scalability,
- horizontal scalability,
- low latency,
- high throughput,

CAP theorem (here we need to prioritise Strong consistency for booking scenario i.e Consistency>>Availability in order to avoid double booking or double payment & for search scenario we can prioritise 99.99% uptime i.e Availability>>Consistency)

Core Entities:

- user,
- booking,
- payment,
- history

API :

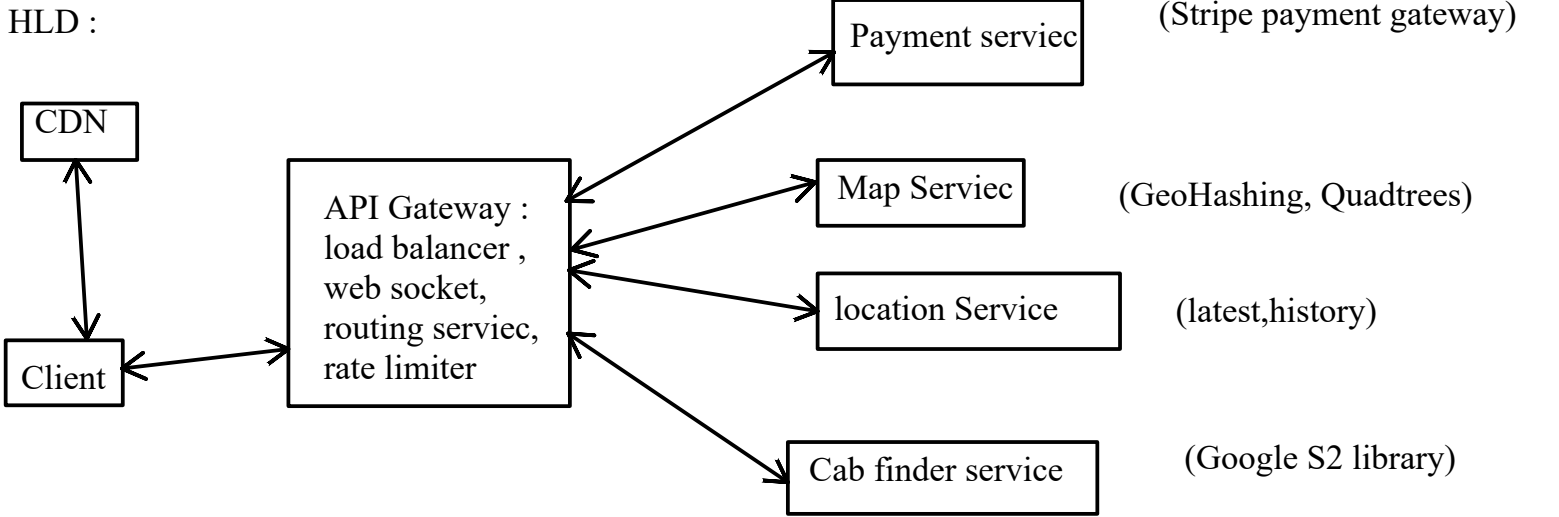
POST:/booking

```
{
  userID,
  bookingID,
  bookingStatus
}
```

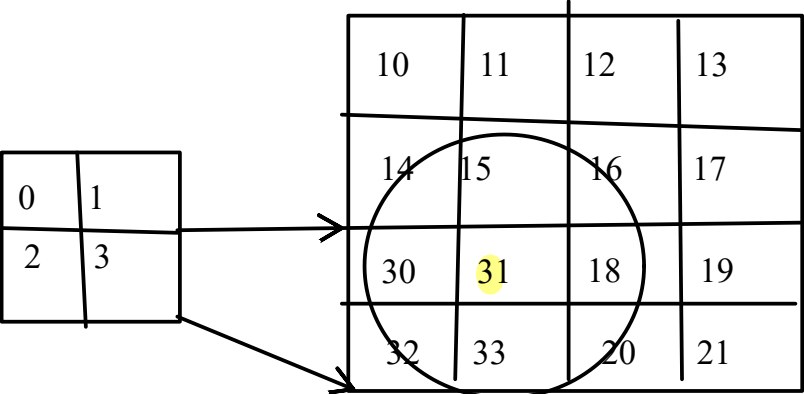
GET:/search/locations/{locationID}

POST:/payment/

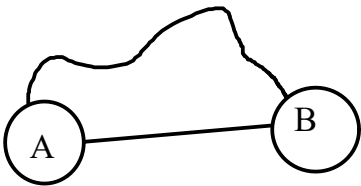
```
{
  userID,
  paymentID,
  paymentStatus
}
```



Map Service
(lat,long) -> chunkID



Cab Finder Service
chunkID->(nodes)



Euclidian distance,
Google S2 library

consistent hashing,
gossip protocol